Test-Driven Development Integration & Anti-Bluff Verification

Revision: 1 **Last modified:** 2026-07-24T00:00:00Z

Table of Contents

1. TDD Philosophy
2. RED Phase Specification
3. GREEN Phase Specification
4. REFACTOR Phase Specification
5. Anti-Bluff Verification
6. Per-Nano-Task Test Template
7. Test Coverage Requirements
8. Integration with Systematic Debugging
9. Constitutional Binding
10. Implementation Gate Source Code

[1]

---

[1] [1]

. TDD Philosophy

. Core Tenet

Test-Driven Development for the SpecKit-Superpowers bridge system follows the immutable cycle:

```
RED ──→ GREEN ──→ REFACTOR ──→ (repeat)
```

Every nano-task — regardless of size, language, or domain — MUST pass through all three phases in strict order. This is not a guideline. It is a mechanical enforcement[1][2] gate implemented by the `implementation-gate.js` file that gates every SpecKit `tasks.md` execution.

. Why TDD for EVERY Nano-Task

The bridge system decomposes user-facing features into sub-sub-sub-N-layer nano-tasks. Each nano-task is intentionally small enough that a modest local LLM can execute it correctly. This extreme granularity would be a BLUFF if tests came after — because:

1. **A task without a test is a claim without evidence** (§11.4.123). If the test is written after the implementation, it proves only that the test agrees with the code — never that the test catches broken behaviour (§11.4.43 PASS-bluff at the TDD layer).
2. **A nano-task without RED-before-GREEN is unvalidated instrumentation** (§11.4.115(F)). The implementation gate must observe the test FAIL before the implementation exists. A guard never observed FAILing does not satisfy the custody chain (§11.4.146(D3)).
3. **Refactoring without a passing test is gambling**[1][3]. The test is the anchor that proves the behaviour is preserved through structural changes.

. Per-Language Binding

TDD applies to ALL executable surfaces in the bridge ecosystem:

Work Product	TDD Artifact
TypeScript/JavaScript (SuperBridge MCP server)	Jest/Vitest `.test.ts`
Go (Helix LLM Gateway, DAG Engine)	`go test` `_test.go` files
Bash (tooling scripts, helpers)	Executing test through the script's REAL invocation path asserting exit + output + state delta (§11.4.224(A))
Configuration (YAML/JSON manifests)	Schema validation test with golden-good/golden-bad fixtures
Pre-build gate	Paired §1.1 mutation authored FIRST, observed to FAIL the gate

Documentation-only changes and governance prose are NOT coverage-scoped (§11.4.224(F)) but MUST pass §11.4.142 independent review.[2]

---

[2] [1]

. RED Phase Specification

. Mechanical Requirements

The RED phase is gated by `implementation-gate.js` with the following invariants:

(a) Test Written FIRST. The test file MUST be authored and committed BEFORE any implementation file for the nano-task. The gate checks file timestamps or git history ordering.

(b) Test Executed FIRST. The test MUST be run before the implementation exists. The gate captures the test runner's exit code and output.

(c) Test Observed to FAIL. The gate REQUIRES a non-zero exit code from the test runner. A test that passes before implementation is a TAUTOLOGY — it does not catch the gap, and the gate refuses it with `RED_FAILED: test passed before implementation — tautology detected`.

(d) Captured Evidence of RED. Every RED phase produces a machine-written verdict file:


```
{
  "item_id": "ATM-512",
  "guard_identity": "test_nano_task_stdlib_sha256",
  "polarity": "RED",
  "exit_code": 1,
  "target_fingerprint": "sha256:abc123def456",
  "iterations": 3,
  "evidence_class": "source",
  "evidence_files": ["qa-results/run-512/RED_stdout.txt", "qa-results/run-512/RED_stderr.txt"],
  "precondition_provenance": "observed",
  "timestamp": "2026-07-24T12:00:00Z"
}
```


. RED MUST NOT Be a Tautology

The implementation gate rejects RED attempts where:

- The test asserts `true === true` or equivalent assertion-free structure.
- The test only checks that a file exists without checking behaviour.
- The test passes with exit 0 before implementation (proves it doesn't test the gap).
- The test's assertion is structurally unable to fail (e.g. `expect(1).toBe(1)`).
- The test's mutation (§1.1 paired mutation) does NOT make the gate FAIL.

Detection mechanism: the gate runs the test in a sandboxed environment where the implementation module is stubbed to throw `NotImplementedError`. If the test does not fail with a meaningful assertion error (not merely a module-not-found error that would exist after implementation too), it is rejected.

. `implementation-gate.js` RED Enforcement

The gate's RED enforcement logic:

```
1. Receive nano-task spec (item_id, test_files[], impl_files[])
2. Verify test files exist and impl files do NOT yet exist
3. Run test runner (jest/vitest/go test/bash)
4. Capture exit code
5. If exit_code === 0 → REJECT "RED_FAILED: tautology — test passed before
implementation"
6. If exit_code !== 0 AND error matches expected pattern → ACCEPT
"RED_PASS"
7. Write RED verdict file to qa-results/<run-id>/verdicts/
RED_<item_id>.json
8. Record in evidence store
```

. RED Verdict Lifecycle

The RED verdict is immutable once written. It serves as the `prior RED` artifact that the GREEN phase gate checks. A GREEN verdict without a matching RED verdict for the same `item_id` is REJECTED by the GREEN-phase gate.

. GREEN Phase Specification

. Mechanical Requirements

(a) Implementation Makes Test Pass. After the implementation file is authored, the test is re-run. The gate captures the exit code: exit 0 with all assertions passing = GREEN.

(b) GREEN REQUIRES Prior RED Artifact. The gate reads `qa-results/<run-id>/verdicts/RED_<item_id>.json`. If it does not exist, the GREEN verdict is REFUSED:

```
GREEN_REJECTED: no prior RED verdict found for item ATM-512.
Every GREEN requires a prior RED observed-to-fail on the pre-
implementation artifact.
TDD must be followed: RED first, GREEN second.
```

(c) Different Artifact Fingerprint Required. The target fingerprint in the GREEN verdict MUST differ from the RED verdict's fingerprint. Identical fingerprints prove the fix was never deployed (§11.4.115(F)). The gate computes `sha256(item_id + impl_file_contents + test_file_contents)` for both phases and asserts they differ.

(d) DETERMINISTIC Consistency (§11.4.50). The GREEN test MUST pass N=3 iterations (N=10 for cycle-validation suites) on the same target with identical exit codes.

. GREEN Verdict Output


```
{
  "item_id": "ATM-512",
  "guard_identity": "test_nano_task_stdlib_sha256",
  "polarity": "GREEN",
  "exit_code": 0,
  "target_fingerprint": "sha256:def789abc012",
  "iterations": 3,
  "evidence_class": "source",
  "evidence_files": ["qa-results/run-512/GREEN_stdout.txt"],
  "prior_red_verdict": "qa-results/run-512/verdicts/RED_ATM-512.json",
  "timestamp": "2026-07-24T12:05:00Z"
}
```

. `implementation-gate.js` GREEN Enforcement

```
1. Receive nano-task spec (item_id, test_files[], impl_files[])
2. Verify impl files now exist
3. Check for prior RED verdict: qa-results/<run-id>/verdicts/
RED_<item_id>.json
4. If absent → REJECT "GREEN_REJECTED: no prior RED verdict"
5. Read RED verdict, extract target_fingerprint
6. Run test runner
7. If exit_code !== 0 → REJECT "GREEN_FAILED: test did not pass"
8. Compute current fingerprint
9. If current_fingerprint === RED_fingerprint → REJECT "GREEN_FAILED:
identical fingerprint — fix not deployed"
10. Run N=3 iterations, assert all exit 0 with identical hashes
11. Write GREEN verdict file
12. Record in evidence store
```

4

---

4 1

. REFACTOR Phase Specification

. Safety Guarantees

The REFACTOR phase allows structural improvements after GREEN:

(a) Test MUST Stay GREEN Through Refactor. Every refactor commit MUST re-run the test. If the test transitions from GREEN to RED during refactor, the refactor introduced a regression and MUST be reverted or fixed.

(b) Permitted Refactor Operations:

- Performance improvements (algorithmic complexity reduction)
- Readability improvements (naming, structure, decomposition)
- Safety improvements (error handling, input validation, bounds checking)
- Memory/resource efficiency (reduced allocations, freed resources)

(c) Forbidden During Refactor:

- Behaviour changes that alter the contract tested by the GREEN phase
- New feature additions (these start a new RED phase)

- Silent weakening of assertions (a §11.4.120 reconciliation violation)

. Refactor Verdict

```
{
  "item_id": "ATM-512",
  "polarity": "REFACTOR",
  "pre_refactor_green_verdict": "qa-results/run-512/verdicts/GREEN_ATM-512.json",
  "post_refactor_exit_code": 0,
  "iterations": 3,
  "evidence_files": ["qa-results/run-512/REFACTOR_stdout.txt"],
  "timestamp": "2026-07-24T12:10:00Z"
}
```

. Anti-Bluff Verification

. Metadata-Only PASS Forbidden

A test whose PASS is based solely on file existence, config presence, grep-hit counts, or "no error" exit codes WITHOUT captured runtime evidence is a §11.4 PASS-bluff. The implementation gate enforces this by requiring that every GREEN verdict carries:

1. **The test's OWN assertion output** (not merely the test runner's exit code).
2. **For user-visible features:** §11.4.107 liveness evidence (frame-advance counter, freeze-detection oracle, per-channel RMS for audio).
3. **For API/infrastructure tasks:** Real round-trip evidence (actual HTTP response, actual DB query result, actual file system state).
4. **For config/static data tasks:** Schema-validated parsing output with golden-good/golden-bad fixtures.

. Self-Validating Oracle

Every test harness that produces a PASS/FAIL verdict MUST be self-validated with:

(a) Golden-Good Fixture. A known-correct artifact that MUST produce PASS. If the oracle fails the golden-good, the ORACLE is broken.

(b) Golden-Bad Fixture. A known-broken artifact that MUST produce FAIL. If the oracle passes the golden-bad, the ORACLE is a BLUFF (§11.4.107(10)).

(c) Negative-Control Fixture. Two genuinely DIFFERENT artifacts that MUST NOT be conflated. An oracle that merges them is a false-merge engine (§11.4.214).

These fixtures are wired into the meta-test ([5] [3] `meta_test_false_positive_proof.sh`) so the oracle's own validity is mechanically verified at every pre-build sweep.

- ### `verify_anti_bluff` MCP Tool

The SuperBridge MCP server exposes a `verify_anti_bluff` tool:

```
// MCP tool: verify_anti_bluff
{
  name: "verify_anti_bluff",
  description: "Verify a nano-task's TDD verdict is anti-bluff compliant per §11.4.115(F) /
                §11.4.146(D3)",
  inputSchema: {
    type: "object",
    properties: {
      item_id:      { type: "string", description: "ATM-NNN or SPK-NNN identifier" },
      red_verdict:  { type: "string", description: "Path to RED verdict JSON" },
      green_verdict:{ type: "string", description: "Path to GREEN verdict JSON" }
    },
    required: ["item_id", "red_verdict", "green_verdict"]
  }
}
```

Implementation logic:

```
async function verify_anti_bluff(args: { item_id: string; red_verdict: string; green_verdict: string }) {
  const red = JSON.parse(await fs.readFile(args.red_verdict, 'utf-8'));
  const green = JSON.parse(await fs.readFile(args.green_verdict, 'utf-8'));

  const findings: string[] = [];

  if (red.exit_code === 0) {
    findings.push("RED_BLUFF: RED test passed — tautology: never observed to fail");
  }
  if (red.target_fingerprint === green.target_fingerprint) {
    findings.push("GREEN_BLUFF: Identical fingerprints — fix was never deployed, GREEN on old
        artifact");
  }
  if (red.item_id !== green.item_id) {
    findings.push("MISMATCH: RED and GREEN verdicts are for different items");
  }
  if (!green.prior_red_verdict) {
    findings.push("MISSING_RED: GREEN verdict lacks prior RED reference");
  }
  if (red.iterations < 3) {
    findings.push("NONDETERMINISTIC: RED executed <3 iterations (§11.4.50)");
  }
  if (green.iterations < 3) {
    findings.push("NONDETERMINISTIC: GREEN executed <3 iterations (§11.4.50)");
  }
  if (red.evidence_files.length === 0 || green.evidence_files.length === 0) {
    findings.push("NO_EVIDENCE: Verdict lacks captured-evidence file paths (§11.4.69)");
  }
  for (const f of [...red.evidence_files, ...green.evidence_files]) {
    try {
      const stat = await fs.stat(f);
      if (stat.size === 0) {
        findings.push(`EMPTY_EVIDENCE: Evidence file ${f} is 0 bytes (§11.4.5)`);
      }
    } catch {
      findings.push(`MISSING_EVIDENCE: Evidence file ${f} not found`);
    }
  }

  return {
    content: [{
      type: "text",
      text: findings.length === 0
        ? `PASS: ${args.item_id} — anti-bluff verified (RED→GREEN, distinct fingerprints, ≥3 iters,
            evidence present)`
```

```
        : `FAIL: ${args.item_id} — ${findings.length} bluff(s) found:\n${findings.map(f => `  - ${f}
            `).join('\n')}`
    }]
  };
}
```

---

6 1

. Per-Nano-Task Test Template

. Go Template

```go
package nano_task

import (
    "testing"
    "os"
    "encoding/json"
)

// TestNanoTask_ATM512_StdlibSHA256 is the TDD guard for nano-task ATM-512.
// RED_MODE=1: asserts the defect/gap is present (no implementation yet).
// RED_MODE=0: asserts the implementation is correct.
func TestNanoTask_ATM512_StdlibSHA256(t *testing.T) {
    redMode := os.Getenv("RED_MODE")
    if redMode == "" {
        redMode = "1" // default: RED — assert broken/absent
    }

    input := "hello world"
    expected := "b94d27b9934d3e08a52e52d7da7dabfac484efe37a5380ee9088f7ace2efcde9"

    result, err := StdlibSHA256(input)

    if redMode == "1" {
        // RED: expecting failure — implementation not yet built
        if err == nil && result == expected {
            t.Fatalf("RED_TAUTOLOGY: implementation returned correct result before RED phase. The
                test is not catching the gap. Is the implementation already present?")
        }
        // RED PASS: implementation absent or broken — the expected state
        t.Logf("RED_PASS: stdlib_sha256 correctly reports broken/absent in RED mode (err=%v,
                result=%q)", err, result)
        return
    }

    // GREEN: implementation must be correct
    if err != nil {
        t.Fatalf("GREEN_FAILED: stdlib_sha256 returned error: %v", err)
    }
    if result != expected {
        t.Fatalf("GREEN_FAILED: stdlib_sha256(%q) = %q, want %q", input, result, expected)
    }
    t.Logf("GREEN_PASS: stdlib_sha256(%q) = %q", input, result)
}
```

. TypeScript Template

```typescript
// test_nano_task_ATM512_stdlib_sha256.test.ts
import { describe, it, expect, beforeAll } from 'vitest';

const RED_MODE = process.env.RED_MODE || '1';

describe('NanoTask ATM-512: StdlibSHA256', () => {
  let sut: typeof import('../src/atm512_stdlib_sha256').StdlibSHA256;

  beforeAll(async () => {
    try {
      const mod = await import('../src/atm512_stdlib_sha256');
      sut = mod.StdlibSHA256;
    } catch {
      sut = undefined as any;
    }
  });

  const input = 'hello world';
  const expected = 'b94d27b9934d3e08a52e52d7da7dabfac484efe37a5380ee9088f7ace2efcde9';

  it('TDD guard: RED=tests defect absence, GREEN=tests implementation', () => {
    if (RED_MODE === '1') {
      // RED phase: implementation should be absent or broken
      if (sut === undefined) {
        // Expected: module not yet created
        console.log('RED_PASS: stdlib_sha256 module not yet implemented');
        return;
      }

      let result: string | undefined;
      let threw = false;
      try {
        result = sut(input);
      } catch {
        threw = true;
      }

      if (!threw && result === expected) {
        throw new Error(
          'RED_TAUTOLOGY: implementation returned correct result before RED phase. ' +
          'The test is not catching the gap. Is the implementation already present?'
        );
      }
      console.log('RED_PASS: stdlib_sha256 correctly reports broken/absent in RED mode');
      return;
```

```
    }

    // GREEN phase: implementation must be correct
    if (sut === undefined) {
      throw new Error('GREEN_FAILED: stdlib_sha256 module not found');
    }
    const result = sut(input);
    expect(result).toBe(expected);
    console.log(`GREEN_PASS: stdlib_sha256("${input}") = "${result}"`);
  });
});
```

. Bash Template

```bash
#!/usr/bin/env bash
set -euo pipefail
# Test: test_nano_task_ATM512_helper_script.sh
# §11.4.224(A): Executing test through REAL invocation path.

RED_MODE="${RED_MODE:-1}"
SCRIPT_UNDER_TEST="${SCRIPT_UNDER_TEST:-./scripts/helpers/atm512_helper.sh}"

if [[ "$RED_MODE" == "1" ]]; then
    # RED: script not yet implemented — expect non-zero or "not found"
    if [[ ! -x "$SCRIPT_UNDER_TEST" ]]; then
        echo "RED_PASS: $SCRIPT_UNDER_TEST does not yet exist or is not executable"
        exit 0
    fi

    set +e
    output=$("$SCRIPT_UNDER_TEST" "test_input" 2>&1)
    rc=$?
    set -e

    if [[ $rc -eq 0 ]]; then
        echo "RED_TAUTOLOGY: $SCRIPT_UNDER_TEST exited 0 — implementation may already
              exist"
        exit 1
    fi
    echo "RED_PASS: $SCRIPT_UNDER_TEST correctly failed (rc=$rc): $output"
    exit 0
fi

# GREEN: script must work correctly
if [[ ! -x "$SCRIPT_UNDER_TEST" ]]; then
    echo "GREEN_FAILED: $SCRIPT_UNDER_TEST not found or not executable"
    exit 1
fi

output=$("$SCRIPT_UNDER_TEST" "test_input" 2>&1)
rc=$?

if [[ $rc -ne 0 ]]; then
    echo "GREEN_FAILED: $SCRIPT_UNDER_TEST exited $rc: $output"
    exit 1
fi

if [[ "$output" != "expected_output" ]]; then
    echo "GREEN_FAILED: expected 'expected_output', got '$output'"
    exit 1
```

```
fi

echo "GREEN_PASS: $SCRIPT_UNDER_TEST correctly produced '$output'"
```

Test Coverage Requirements

Minimum Coverage Floor

Per §11.4.224(B): **≥85% line coverage, ~100% target**, applying to ALL executable code in the bridge ecosystem:

Language	Instrument	Notes
Go	`go test -coverprofile` + `go tool cover`	Built-in, measures per-function and per-line
TypeScript	`vitest --coverage` (c8/istanbul)	Branch and line coverage
Bash	`PS4='+COV:${BASH_SOURCE##*/}:${LINENO}:' bash -x`	Line coverage only; branch coverage needs `kcov`
Config (YAML/JSON)	Schema validation with golden fixtures	Full schema coverage, no uncovered properties

RED-Capable Tests Only

Per the §11.4.224 amended contract: the measured numerator counts ONLY RED-CAPABLE tests — every contributing test carries a recorded failing-first run OR a paired mutation observed to make it FAIL. Zero-assertion contributors are REJECTED by construction. A single RED-proven test plus 84% assertion-free padding → gate FAILs.

. Exclusion List

Corpus exclusions are legal ONLY via a CHECKED-IN exclusion list, each entry justified from the closed class set:

- `generated-code` — output of a committed generator per §11.4.77
- `vendored-third-party` — unmodified third-party libraries
- `non-shipping-fixtures-and-golden-assets` — test fixtures, golden files

First-party exclusions additionally REQUIRE a tracked §11.4.197 item.
[8]

---

[8] [1] . Integration with Systematic Debugging

. Auto-Trigger on RED Failure

When a RED test fails UNEXPECTEDLY (i.e., the test was supposed to fail in a specific way but failed differently), the `implementation-gate.js` auto-triggers `superpowers:systematic-debugging` per §11.4.102(D):

```
async function handleRedFailure(item_id: string, redVerdict: RedVerdict, testOutput: string) {
  const expectedFailurePattern = extractExpectedPattern(redVerdict);
  const actualFailure = testOutput;

  if (!actualFailure.match(expectedFailurePattern)) {
    // UNEXPECTED RED failure — not the anticipated gap
    console.log(`§11.4.102(D): Auto-activating systematic-debugging for ${item_id}`);

    await dispatchSystematicDebugging({
      item_id,
      phase: "RED",
      expected: expectedFailurePattern,
      actual: actualFailure,
      red_verdict: redVerdict
    });
  }
}
```

. Debugging Integration Contract

When systematic-debugging is auto-activated:

1. The current TDD pipeline PAUSES the nano-task (marks it `crashed` / `in-flight` per §11.4.147).
2. Systematic-debugging investigates: Is the test wrong? Is the gap description wrong? Is there a deeper defect?
3. The nano-task is RESUMED only after debugging produces a CLEAN root-cause finding.
4. A spurious failure without root cause → re-enters the §11.4.114/.115 isolate→RED→fix loop.

[9]

. Constitutional Binding

This TDD integration document binds to the following constitution anchors:

Anchor	What It Mandates	How This Document Satisfies It
§11.4.43	TDD-fix-discipline: RED→LIVE-ADB→GREEN→VERIFY→DOCUMENT	All five steps mechanized in `implementation-gate.js`
§11.4.115	RED-baseline-on-the-broken-artifact + polarity switch	`RED_MODE` env var, RED-then-GREEN verdict chain
§11.4.224	TDD for ALL work, not only fixes; ≥85% coverage floor	Coverage gate wired; bash scripts included
§11.4.146	Reproduce-first test + same-test-confirms-fix + extend-to-all-cases	RED=STEP1, GREEN=STEP2, fan-out mandate = STEP3
§11.4.50	Deterministic consistency: N=3 iterations, identical exit codes	Enforced in GREEN phase gate
§11.4.69	Every PASS cites captured-evidence artifact path	Embedded in verdict JSON schema
§11.4.1	FAIL-bluffs forbidden — test crashes counted as findings	Gate rejects exit-code-based-only PASS detection
§11.4.6	No-guessing mandate	All verdict fields machine-derived, never hand-typed
§11.4.107(10)	Self-validated analyzer with golden-good/golden-bad fixtures	`verify_anti_bluff` oracle self-tests

Anchor	What It Mandates	How This Document Satisfies It
§11.4.108	Four-layer fix-verification + runtime-signature-as-done	Target fingerprint comparison in GREEN phase
§11.4.102(D)	Automatic systematic-debugging activation	Auto-triggered on unexpected RED failure
§11.4.147	Crashed-agent respawn-until-complete	Nano-task pause/resume on debugging activation
§11.4.135	Standing regression-guard suite	Every nano-task's GREEN becomes a permanent guard
§11.4.146(D3)	Status-custody: done-status requires RED+GREEN verdict pair	Gate refuses GREEN without prior RED

. Complete `implementation-gate.js` Source Code

```
#!/usr/bin/env node

/**
 * implementation-gate.js
 * Purpose:  Mechanically enforce TDD (RED → GREEN → REFACTOR) for every
 *           SpecKit nano-task in the SpecKit-Superpowers bridge system.
 * Usage:    node implementation-gate.js <command> [args...]
 * Commands:
 *   red    <item_id> <test_files...>     — Run RED phase gate
 *   green  <item_id> <test_files...> <impl_files...> — Run GREEN phase gate
 *   refactor <item_id> <impl_files...> <test_files...> — Run REFACTOR phase gate
 *   verify <item_id>                     — Run anti-bluff verification
 *   status <item_id>                     — Show current TDD state for item
 *
 * Constitutional anchors: §11.4.43, §11.4.115, §11.4.146, §11.4.224, §1.1
 */

const fs = require('fs');
const path = require('path');
const crypto = require('crypto');
const { execSync, exec } = require('child_process');

// ——— Configuration ———————————————————————————————————————————————————————

const CONFIG = {
  verdictDir: process.env.TDD_VERDICT_DIR || 'qa-results/tdd_verdicts',
  evidenceDir: process.env.TDD_EVIDENCE_DIR || 'qa-results/tdd_evidence',
  deterministicIterations: parseInt(process.env.TDD_ITERATIONS || '3', 10),
  maxIterations: 10,
  coverageThreshold: parseFloat(process.env.TDD_COVERAGE_THRESHOLD || '0.85'),
  testRunner: {
    '.ts': 'npx vitest run --reporter=json',
    '.js': 'npx jest --json',
    '.go': 'go test -v -count=1',
    '.sh': 'bash -euo pipefail'
  },
  notImplementedStub: 'throw new Error("NOT_IMPLEMENTED: nano-task not yet built");'
};

// ——— Helpers ——————————————————————————————————————————————————————————————

function sha256(data) {
```

```
  return crypto.createHash('sha256').update(data).digest('hex');
}

function timestamp() {
  return new Date().toISOString();
}

function ensureDir(dir) {
  if (!fs.existsSync(dir)) {
    fs.mkdirSync(dir, { recursive: true });
  }
}

function readJSON(filePath) {
  if (!fs.existsSync(filePath)) return null;
  return JSON.parse(fs.readFileSync(filePath, 'utf-8'));
}

function writeJSON(filePath, data) {
  ensureDir(path.dirname(filePath));
  fs.writeFileSync(filePath, JSON.stringify(data, null, 2) + '\n');
}

function computeFingerprint(itemId, files) {
  const contents = files
    .filter(f => fs.existsSync(f))
    .map(f => fs.readFileSync(f, 'utf-8'))
    .join('');
  return sha256(itemId + contents);
}

function detectLanguage(testFiles) {
  for (const f of testFiles) {
    if (f.endsWith('_test.go')) return '.go';
    if (f.endsWith('.test.ts') || f.endsWith('.test.tsx')) return '.ts';
    if (f.endsWith('.test.js') || f.endsWith('.test.jsx')) return '.js';
    if (f.endsWith('.sh')) return '.sh';
  }
  // fallback: check file content for shebang
  for (const f of testFiles) {
    if (fs.existsSync(f)) {
      const head = fs.readFileSync(f, 'utf-8').slice(0, 100);
      if (head.startsWith('#!/usr/bin/env bash') || head.startsWith('#!/bin/bash')) return '.sh';
    }
  }
  return '.ts'; // default
```

```
}

function runTest(testFiles, env = {}) {
  const lang = detectLanguage(testFiles);
  const cmd = CONFIG.testRunner[lang] || CONFIG.testRunner['.ts'];
  const testArg = testFiles.join(' ');

  const fullCmd = `${cmd} ${testArg}`;

  try {
    const output = execSync(fullCmd, {
      encoding: 'utf-8',
      env: { ...process.env, ...env },
      stdio: 'pipe',
      maxBuffer: 10 * 1024 * 1024
    });
    return { exitCode: 0, output };
  } catch (err) {
    return {
      exitCode: err.status || 1,
      output: (err.stdout || '') + '\n' + (err.stderr || '')
    };
  }
}

function isTautology(testFiles) {
  // Heuristic: scan test files for assertion-free structure
  for (const f of testFiles) {
    if (!fs.existsSync(f)) continue;
    const content = fs.readFileSync(f, 'utf-8');

    // Check for common tautology patterns
    const tautologyPatterns = [
      /expect\(true\)\.toBe\(true\)/,
      /expect\(1\)\.toBe\(1\)/,
      /assert\.Equal\(t,\s*true,\s*true\)/,
      /t\.Errorf?\("always pass"/i,
      /\/\/ always pass/i,
      /_mutated_.*pass/i
    ];

    for (const pattern of tautologyPatterns) {
      if (pattern.test(content)) {
        return { isTautology: true, reason: `Tautology pattern found: ${pattern}` };
      }
    }
```

```
  }
  return { isTautology: false, reason: null };
}

// ——— Sandboxed Tautology Detection ————————————————————————

function runSandboxedRedTest(testFiles, itemId) {
  /**
   * Create a sandbox environment where the implementation module
   * is stubbed. If the test still passes, it's a tautology.
   */
  const sandboxDir = `/tmp/tdd_sandbox_${itemId}_${Date.now()}`;
  fs.mkdirSync(sandboxDir, { recursive: true });

  try {
    // Copy test files to sandbox
    for (const f of testFiles) {
      const dest = path.join(sandboxDir, path.basename(f));
      fs.copyFileSync(f, dest);
    }

    // Create a stub implementation that throws
    const lang = detectLanguage(testFiles);
    let stubPath;
    if (lang === '.ts' || lang === '.js') {
      stubPath = path.join(sandboxDir, 'stub_impl.ts');
      fs.writeFileSync(stubPath, CONFIG.notImplementedStub);
    } else if (lang === '.go') {
      stubPath = path.join(sandboxDir, 'stub_impl.go');
      fs.writeFileSync(stubPath, `package nano_task\n\nfunc StubImpl() string
            {\n\tpanic("NOT_IMPLEMENTED")\n}\n`);
    }

    // Run test in sandbox
    const result = runTest(
      testFiles.map(f => path.join(sandboxDir, path.basename(f))),
      { RED_MODE: '1', TDD_SANDBOX: '1' }
    );

    return {
      exitCode: result.exitCode,
      output: result.output,
      isTautology: result.exitCode === 0
    };
  } finally {
    // Cleanup sandbox
```

```
    try { fs.rmSync(sandboxDir, { recursive: true }); } catch {}
  }
}

// ——— RED Phase ————————————
```

```
  };
}

// 5. Run real RED test (no implementation yet → should fail)
console.log('  Running real RED test...');
const realResult = runTest(testFiles, { RED_MODE: '1' });

if (realResult.exitCode === 0) {
  return {
    verdict: 'REJECTED',
    reason: 'RED_FAILED: Test exited 0 in RED mode — tautology. The test was not observed to fail
            on the broken artifact. Verify that the implementation is genuinely absent and the test
            genuinely catches the gap.'
  };
}

// 6. Run N iterations for deterministic consistency
console.log(`  Running ${CONFIG.deterministicIterations} iterations for deterministic consistency...`);
const iterations = [];
for (let i = 0; i < CONFIG.deterministicIterations; i++) {
  const iterResult = runTest(testFiles, { RED_MODE: '1' });
  iterations.push({ iteration: i + 1, exitCode: iterResult.exitCode, outputHash:
            sha256(iterResult.output) });
}

const exitCodes = new Set(iterations.map(i => i.exitCode));
if (exitCodes.size !== 1) {
  return {
    verdict: 'REJECTED',
    reason: `RED_FAILED: Nondeterministic exit codes across ${CONFIG.deterministicIterations}
            iterations: ${[...exitCodes].join(', ')} (§11.4.50)`
  };
}

// 7. Write RED verdict
const fingerprint = computeFingerprint(itemId, testFiles);
const redVerdict = {
  item_id: itemId,
  guard_identity: `tdd_gate_${itemId}`,
  polarity: 'RED',
  exit_code: realResult.exitCode,
  target_fingerprint: fingerprint,
  iterations: CONFIG.deterministicIterations,
  iteration_exit_codes: iterations.map(i => i.exitCode),
  evidence_class: 'source',
  evidence_files: [],
  precondition_provenance: 'observed',
```

```
    test_output: realResult.output.slice(0, 2000),
    timestamp: timestamp()
  };

  const evidencePath = path.join(CONFIG.evidenceDir, itemId, `RED_${itemId}_stdout.txt`);
  ensureDir(path.dirname(evidencePath));
  fs.writeFileSync(evidencePath, realResult.output);
  redVerdict.evidence_files.push(evidencePath);

  const verdictPath = path.join(CONFIG.verdictDir, `RED_${itemId}.json`);
  writeJSON(verdictPath, redVerdict);

  console.log(`  RED_PASS: Verdict written to ${verdictPath}`);
  console.log(`  Evidence: ${evidencePath}`);

  return { verdict: 'ACCEPTED', redVerdict, verdictPath };
}

// ——— GREEN Phase ————————————
```

```
if (!redVerdict) {
  return {
    verdict: 'REJECTED',
    reason: `GREEN_REJECTED: No prior RED verdict found at ${redVerdictPath}. Every GREEN
            requires a prior RED observed-to-fail on the pre-implementation artifact. TDD must be
            followed: RED first, GREEN second.`
  };
}

if (redVerdict.exit_code === 0) {
  return {
    verdict: 'REJECTED',
    reason: 'GREEN_REJECTED: Prior RED verdict shows exit code 0 — RED was never observed
            to fail. The RED phase was a tautology. Re-run RED phase with implementation genuinely
            absent.'
  };
}

// 4. Run GREEN test
console.log('  Running GREEN test...');
const result = runTest(testFiles, { RED_MODE: '0' });

if (result.exitCode !== 0) {
  return {
    verdict: 'REJECTED',
    reason: `GREEN_FAILED: Test exited ${result.exitCode}. The implementation did not make the
            test pass.\nOutput:\n${result.output.slice(0, 2000)}`
  };
}

// 5. Compute current fingerprint, compare vs RED
const currentFingerprint = computeFingerprint(itemId, [...testFiles, ...implFiles]);

if (currentFingerprint === redVerdict.target_fingerprint) {
  return {
    verdict: 'REJECTED',
    reason: 'GREEN_FAILED: Identical fingerprints — the artifact has not changed since RED. The
            fix was never deployed. (§11.4.115(F))'
  };
}

// 6. Run N iterations for deterministic consistency
console.log(`  Running ${CONFIG.deterministicIterations} iterations for deterministic consistency...`);
const iterations = [];
for (let i = 0; i < CONFIG.deterministicIterations; i++) {
  const iterResult = runTest(testFiles, { RED_MODE: '0' });
  iterations.push({ iteration: i + 1, exitCode: iterResult.exitCode, outputHash:
            sha256(iterResult.output) });
```

```
  }

  const exitCodes = new Set(iterations.map(i => i.exitCode));
  if (exitCodes.size !== 1 || [...exitCodes][0] !== 0) {
    return {
      verdict: 'REJECTED',
      reason: `GREEN_FAILED: Nondeterministic: ${exitCodes.size > 1 ? 'varying exit codes' : 'exit
              code not 0'} across ${CONFIG.deterministicIterations} iterations (§11.4.50)`
    };
  }

  // 7. Write GREEN verdict
  const greenVerdict = {
    item_id: itemId,
    guard_identity: `tdd_gate_${itemId}`,
    polarity: 'GREEN',
    exit_code: 0,
    target_fingerprint: currentFingerprint,
    iterations: CONFIG.deterministicIterations,
    evidence_class: 'source',
    evidence_files: [],
    prior_red_verdict: redVerdictPath,
    test_output: result.output.slice(0, 2000),
    timestamp: timestamp()
  };

  const evidencePath = path.join(CONFIG.evidenceDir, itemId, `GREEN_${itemId}_stdout.txt`);
  ensureDir(path.dirname(evidencePath));
  fs.writeFileSync(evidencePath, result.output);
  greenVerdict.evidence_files.push(evidencePath);

  const verdictPath = path.join(CONFIG.verdictDir, `GREEN_${itemId}.json`);
  writeJSON(verdictPath, greenVerdict);

  console.log(`  GREEN_PASS: Verdict written to ${verdictPath}`);

  return { verdict: 'ACCEPTED', greenVerdict, verdictPath };
}

// ——— REFACTOR Phase ————————————
```

```
  const greenVerdictPath = path.join(CONFIG.verdictDir, `GREEN_${itemId}.json`);
  const greenVerdict = readJSON(greenVerdictPath);

  if (!greenVerdict) {
    return {
      verdict: 'REJECTED',
      reason:
        `REFACTOR_REJECTED: No prior GREEN verdict found. Refactoring requires a passing
        GREEN test first.`
    };
  }

  // 2. Run tests — must stay GREEN
  console.log('  Running tests after refactor...');
  const result = runTest(testFiles, { RED_MODE: '0' });

  if (result.exitCode !== 0) {
    return {
      verdict: 'REJECTED',
      reason: `REFACTOR_FAILED: Tests no longer pass after refactor. Exit ${result.exitCode}. The
        refactor introduced a regression.\nOutput:\n${result.output.slice(0, 2000)}`
    };
  }

  // 3. Write REFACTOR verdict
  const refactorVerdict = {
    item_id: itemId,
    polarity: 'REFACTOR',
    pre_refactor_green_verdict: greenVerdictPath,
    post_refactor_exit_code: 0,
    iterations: 1,
    evidence_files: [],
    timestamp: timestamp()
  };

  const evidencePath = path.join(CONFIG.evidenceDir, itemId, `REFACTOR_${itemId}_stdout.txt`);
  ensureDir(path.dirname(evidencePath));
  fs.writeFileSync(evidencePath, result.output);
  refactorVerdict.evidence_files.push(evidencePath);

  const verdictPath = path.join(CONFIG.verdictDir, `REFACTOR_${itemId}.json`);
  writeJSON(verdictPath, refactorVerdict);

  console.log(`  REFACTOR_PASS: Verdict written to ${verdictPath}`);

  return { verdict: 'ACCEPTED', refactorVerdict, verdictPath };
}
```

```
// ——— Anti-Bluff Verification ———————————————————————————

async function gateVerify(itemId) {
  console.log(`\n[VERIFY] Item: ${itemId}`);

  const redVerdictPath = path.join(CONFIG.verdictDir, `RED_${itemId}.json`);
  const greenVerdictPath = path.join(CONFIG.verdictDir, `GREEN_${itemId}.json`);

  const red = readJSON(redVerdictPath);
  const green = readJSON(greenVerdictPath);

  const findings = [];

  // Check RED verdict
  if (!red) {
    findings.push('MISSING_RED: No RED verdict found');
  } else {
    if (red.exit_code === 0) {
      findings.push('RED_BLUFF: RED test passed — tautology: never observed to fail (§11.4.115)');
    }
    if (red.iterations < CONFIG.deterministicIterations) {
      findings.push(`NONDETERMINISTIC: RED only ${red.iterations} iterations, need ≥${
          CONFIG.deterministicIterations} (§11.4.50)`);
    }
    if (!red.evidence_files || red.evidence_files.length === 0) {
      findings.push('NO_EVIDENCE: RED verdict lacks captured-evidence paths (§11.4.69)');
    }
  }

  // Check GREEN verdict
  if (!green) {
    findings.push('MISSING_GREEN: No GREEN verdict found');
  } else {
    if (!green.prior_red_verdict) {
      findings.push('MISSING_RED_REF: GREEN verdict lacks prior_red_verdict field
          (§11.4.146(D3))');
    }
    if (red && green.target_fingerprint === red.target_fingerprint) {
      findings.push('GREEN_BLUFF: Identical RED/GREEN fingerprints — fix never deployed
          (§11.4.115(F))');
    }
    if (green.exit_code !== 0) {
      findings.push(`GREEN_FAILED: GREEN test exited ${green.exit_code}`);
    }
    if (green.iterations < CONFIG.deterministicIterations) {
```

```
    findings.push(`NONDETERMINISTIC: GREEN only ${green.iterations} iterations, need ≥${CONFIG.deterministicIterations} (§11.4.50)`);
  }
}

// Check evidence files exist and are non-empty
for (const verdict of [red, green].filter(Boolean)) {
  for (const f of (verdict.evidence_files || [])) {
    try {
      const stat = fs.statSync(f);
      if (stat.size === 0) {
        findings.push(`EMPTY_EVIDENCE: ${f} is 0 bytes (§11.4.5)`);
      }
    } catch {
      findings.push(`MISSING_EVIDENCE: ${f} not found`);
    }
  }
}

// Check cross-verdict consistency
if (red && green && red.item_id !== green.item_id) {
  findings.push(`MISMATCH: RED item ${red.item_id} ≠ GREEN item ${green.item_id}`);
}

// Verify evidence class matches defect layer
if (green && green.evidence_class === 'source') {
  // Source-class evidence is valid for source-layer defects (e.g., compilation, lint)
  // but would be WRONG-LAYER for user-visible features
  // This is a WARN not a FAIL — the item's own defect layer determines validity
}

const evidencePath = path.join(CONFIG.evidenceDir, itemId, `VERIFY_${itemId}.json`);
const verifyResult = {
  item_id: itemId,
  findings,
  verdict: findings.length === 0 ? 'PASS' : 'FAIL',
  timestamp: timestamp()
};
writeJSON(evidencePath, verifyResult);

if (findings.length === 0) {
  console.log(`  VERIFY_PASS: ${itemId} — anti-bluff verified (RED→GREEN, distinct fingerprints, ≥${CONFIG.deterministicIterations} iters, evidence present)`);
} else {
  console.log(`  VERIFY_FAIL: ${itemId} — ${findings.length} bluff(s) found:`);
  for (const f of findings) {
    console.log(`    - ${f}`);
```

```
    }
  }

  return verifyResult;
}

// ——— Status ———————————————
```

```
  console.error('  node implementation-gate.js status ATM-512');
  process.exit(1);
}

ensureDir(CONFIG.verdictDir);
ensureDir(CONFIG.evidenceDir);

const remaining = args.slice(2);

let result;
switch (command) {
  case 'red': {
    // Separate test and impl files by convention: test files contain 'test' or '_test'
    const testFiles = remaining.filter(f => f.includes('test') || f.includes('_test') || f.includes('.spec.'));
    const implFiles = remaining.filter(f => !testFiles.includes(f));
    result = await gateRed(itemId, testFiles.length > 0 ? testFiles : remaining, implFiles);
    break;
  }
  case 'green': {
    const testFiles = remaining.filter(f => f.includes('test') || f.includes('_test') || f.includes('.spec.'));
    const implFiles = remaining.filter(f => !testFiles.includes(f));
    result = await gateGreen(itemId, testFiles, implFiles);
    break;
  }
  case 'refactor': {
    const testFiles = remaining.filter(f => f.includes('test') || f.includes('_test') || f.includes('.spec.'));
    const implFiles = remaining.filter(f => !testFiles.includes(f));
    result = await gateRefactor(itemId, implFiles, testFiles);
    break;
  }
  case 'verify':
    result = await gateVerify(itemId);
    break;
  case 'status':
    result = await gateStatus(itemId);
    break;
  default:
    console.error(`Unknown command: ${command}`);
    process.exit(1);
}

if (result.verdict === 'REJECTED') {
  console.error(`\n✗ ${result.verdict}: ${result.reason}`);
  process.exit(2);
} else if (result.verdict === 'FAIL') {
  console.error(`\n✗ VERIFY_FAIL: ${result.findings.length} finding(s)`);
```

```
    process.exit(3);
  }
}

main().catch(err => {
  console.error('FATAL:', err.message);
  process.exit(99);
});
```

End of TDD Integration & Anti-Bluff Verification specification.